PROJECT REPORT

INTELLIGENT SUPPLY CHAIN NETWORK OPTIMIZATION SYSTEM

Design and Analysis of Algorithms

PROJECT DETAILS	INFORMATION
Student Name	PRIYADHARSHINI S
Register Number	711525BCS125
Department / Class	CSE - A
Institution	KALAIGNARKARUNANIDHI INSTITUTE OF TECHNOLOGY
Course	DESIGN ANALYSIS OF ALGORITHMS
Faculty / Guide	Dr. SENTHILKUMAR
Academic Year	2026–2027

Submitted in partial fulfilment of the requirements for the course

DESIGN AND ANALYSIS OF ALGORITHMS

1. INTRODUCTION AND PROBLEM STATEMENT

A supply chain links suppliers, warehouses and customers. Choosing expensive routes can increase delivery costs. This project represents locations as a graph and compares route costs to select a lower-cost path.

Problem: Given route costs and customer demand, identify the cheaper feasible route and estimate total transportation cost.

Objective: Compare routes, calculate cost and demonstrate algorithm-based distribution planning.

2. JAVA PROGRAM

The program compares two sample routes. Supplier–W1 costs 4 + 3 per unit; Supplier–W2 costs 6 + 2 per unit. Demand is 20 units.

```
public class SupplyChainOptimizer {
  public static void main(String[] args) {
    int routeW1 = 4 + 3;
    int routeW2 = 6 + 2;
    int demand = 20;
    int best = Math.min(routeW1, routeW2);
    String route = (routeW1 <= routeW2)
        ? "Supplier -> W1 -> Customer"
        : "Supplier -> W2 -> Customer";
    System.out.println("Best route: " + route);
    System.out.println("Unit cost: " + best);
    System.out.println("Demand: " + demand);
    System.out.println("Total cost: " + best * demand);
  }
}
```

PROGRAM OUTPUT

```
C:\> java SupplyChainOptimizer

Best route: Supplier -> W1 -> Customer
Unit cost: 7
Demand: 20
Total cost: 140

Process finished successfully.
```

3. PROPOSED SYSTEM

The supply chain is stored as a weighted graph: locations are vertices and transport links are edges. Each edge stores its transportation cost.

Working Steps

1. Enter locations, route costs and customer demand.
2. Build the graph from transport links.
3. Compare costs and select the lowest-cost route.
4. Check supply and capacity against demand.
5. Display the selected route and estimated cost.

Algorithms Used

Binary Search: Finds records in sorted data in O(log n).
Merge Sort: Organizes shipment records in O(n log n).
Dijkstra's Algorithm: Finds shortest paths when route costs are non-negative.
Greedy Choice: Selects the lowest-cost feasible option at each step.

Advantages

• Makes route comparison clear and repeatable.
• Helps estimate transportation cost.
• Can be extended to include stock and vehicle capacity.

4. ALGORITHM AND COMPLEXITY

Simplified Route Selection

```
1. Read available route costs.
2. Compare candidate costs.
3. Select the minimum-cost route.
4. Multiply unit cost by demand.
5. Display route and total cost.
```

Complexity Summary

Algorithm	Time Complexity	Use
Linear Search	O(n)	Unsorted record lookup
Binary Search	O(log n)	Sorted record lookup
Merge Sort	O(n log n)	Sort shipment data
Dijkstra (priority queue)	O((V + E) log V)	Shortest path in graph

Sample Calculation

Route W1 = 4 + 3 = 7 per unit. Route W2 = 6 + 2 = 8 per unit. W1 is cheaper. Total estimated cost = 7 × 20 = 140 cost units.

V is the number of locations, E the number of transport links and n the number of records. Actual performance depends on the complete network and implementation.

5. RESULT AND CONCLUSION

Result

The Java program selects Supplier → W1 → Customer because its unit cost is 7, compared with 8 for W2. For demand of 20 units, the estimated cost is 140.

Limitations

This is a small demonstration with two routes and sample values. It does not use live stock, vehicle tracking or real-time transport prices. Accuracy depends on the input data.

Future Enhancement

The system can be expanded for multiple suppliers, warehouses and customers, live inventory, vehicle capacity, delivery deadlines and a graphical user interface.

Conclusion

The project demonstrates how graph-based route selection and algorithm analysis can support cost-aware supply-chain planning. Further development can handle larger networks and practical constraints.

— THANK YOU —